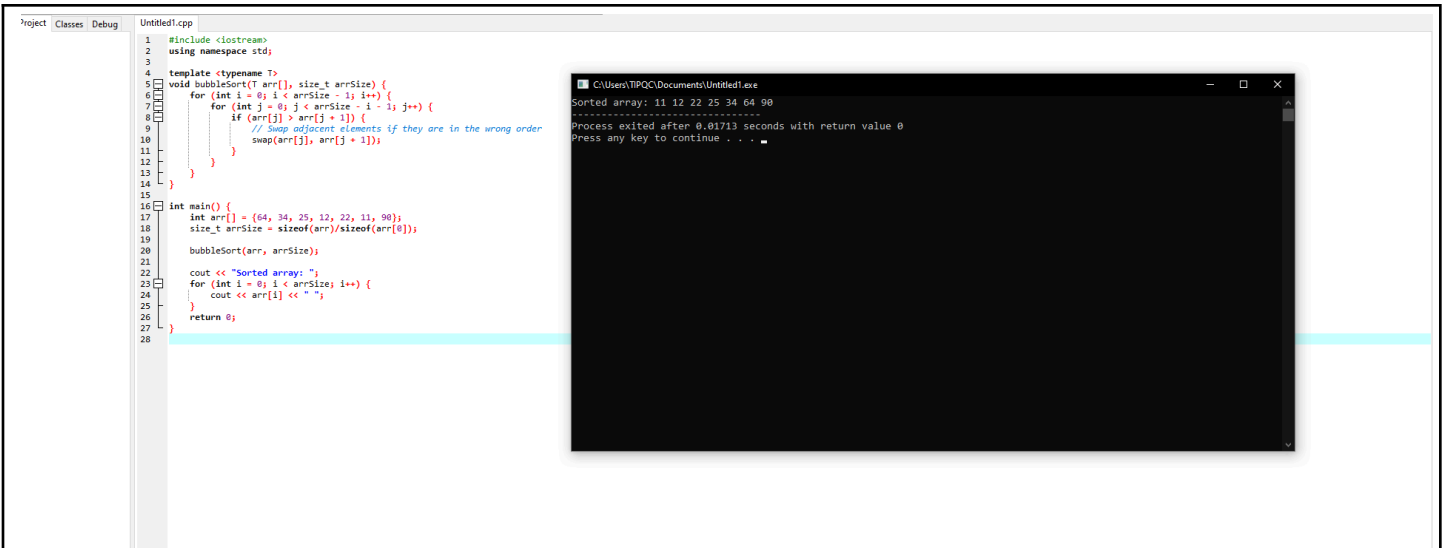
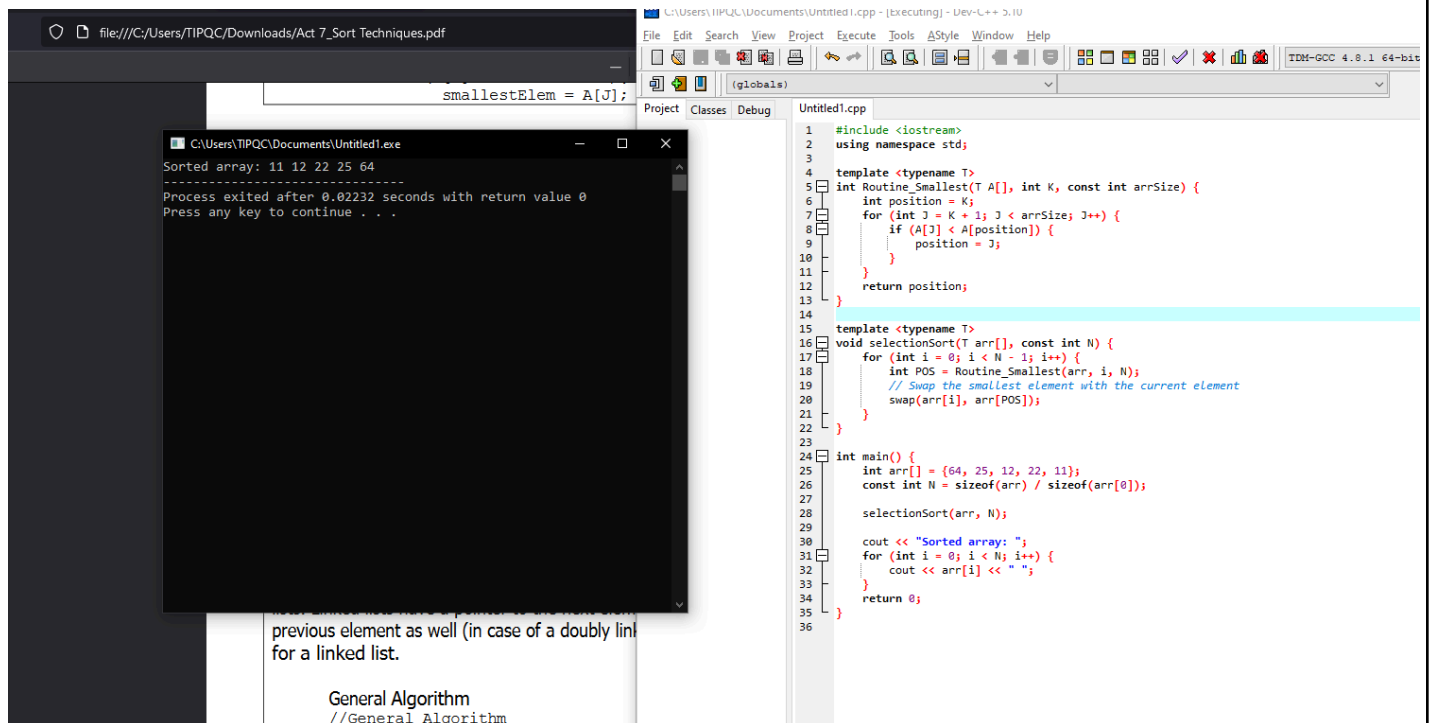




A.2 Selection Sort Algorithm

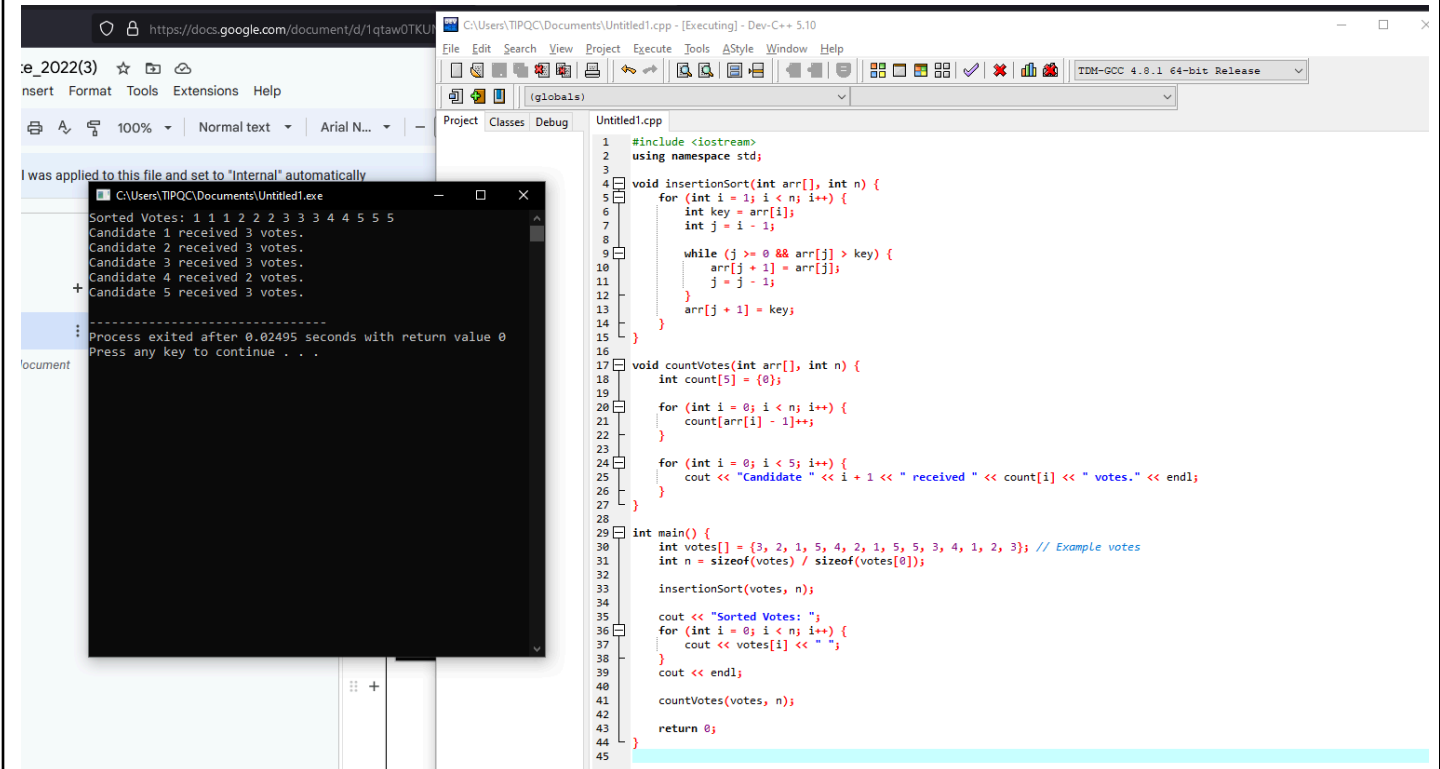


A.3

```
C:\Users\TIPOC\Documents\Untitled1.exe
Unsorted array: 474 391 256 35 1 948 305 494 569 763 740 371 142 55 114 8 731 346 607 167 11 216 856 473 815 642 369 254 503 737 226 863 860 928 27 535 77 31 549 287 0 59
8 11 202 668 943 873 465 236 312 739 871 703 175 778 623 534 158 747 845 437 105 968 402 557 997 434 980 625 572 39 477 888 654 625 979 355 650 960 816 560 917 201 611 37
4 698 566 103 393 316 920 426 576 720 466 112 957 685 480 473
Array after Insertion Sort: 0 1 8 11 11 27 31 35 39 55 77 103 105 112 114 142 158 167 175 201 202 216 226 236 254 256 287 305 312 316 346 355 369 371 374 391 393 402 426
434 437 465 466 473 473 474 477 480 494 503 534 535 549 557 560 566 569 572 576 598 607 611 623 625 625 642 650 654 668 685 698 703 720 731 737 739 740 747 763 778 815 81
6 845 856 860 863 871 873 888 917 920 928 943 948 957 960 968 979 980 997

-----
Process exited after 0.04935 seconds with return value 0
Press any key to continue . . .
```

7. Supplementary Activity



The screenshot displays a development environment with three main components:

- Google Docs:** A browser window showing a document titled "e_2022(3)". The document content includes the text "I was applied to this file and set to 'Internal' automatically." and a list of votes: "Sorted Votes: 1 1 1 2 2 3 3 3 4 4 5 5 5". Below this, it lists "Candidate 1 received 3 votes.", "Candidate 2 received 3 votes.", "Candidate 3 received 3 votes.", "Candidate 4 received 2 votes.", and "Candidate 5 received 3 votes.".
- Terminal Window:** A window titled "C:\Users\TIPOC\Documents\Untitled1.exe" showing the output of the program, which matches the content of the Google Docs window.
- C++ Code Editor:** A window titled "C:\Users\TIPOC\Documents\Untitled1.cpp" showing the source code. The code includes headers for `<iostream>` and `using namespace std;`. It defines two functions: `insertionSort` and `countVotes`. The `main` function initializes an array of votes, sorts it using `insertionSort`, and then prints the sorted votes and the number of votes for each candidate using `countVotes`.

```
1 #include <iostream>
2 using namespace std;
3
4 void insertionSort(int arr[], int n) {
5     for (int i = 1; i < n; i++) {
6         int key = arr[i];
7         int j = i - 1;
8
9         while (j >= 0 && arr[j] > key) {
10             arr[j + 1] = arr[j];
11             j = j - 1;
12         }
13         arr[j + 1] = key;
14     }
15 }
16
17 void countVotes(int arr[], int n) {
18     int count[5] = {0};
19
20     for (int i = 0; i < n; i++) {
21         count[arr[i] - 1]++;
22     }
23
24     for (int i = 0; i < 5; i++) {
25         cout << "Candidate " << i + 1 << " received " << count[i] << " votes." << endl;
26     }
27 }
28
29 int main() {
30     int votes[] = {3, 2, 1, 5, 4, 2, 1, 5, 5, 3, 4, 1, 2, 3}; // Example votes
31     int n = sizeof(votes) / sizeof(votes[0]);
32
33     insertionSort(votes, n);
34
35     cout << "Sorted Votes: ";
36     for (int i = 0; i < n; i++) {
37         cout << votes[i] << " ";
38     }
39     cout << endl;
40
41     countVotes(votes, n);
42
43     return 0;
44 }
```

8. Conclusion

In this task, we had coded an array of 100 elements, with values ranging from 1 to 5. Additionally, we used a sorting algorithm that arranged the number and votes accordingly and then counted how many times each candidate represented by the numbers received a vote. The sorting method made it easier to comprehend the code allowing the coder or the reader to fully understand what the code is all about.

9. Assessment Rubric